



Universidad autónoma de baja california

Ingeniería en computación

Sistemas embebidos

Reporte de laboratorio 1

Evangelina Lara Camacho

Erik garcia chavez 01275863

5 de septiembre del 2025

- Teoría: describa la función de *gpio_dump_io_configuration*

Esta función nos proporciona información detallada sobre la configuración actual de todos los pines GPIO del microcontrolador.

Tanto especificaciones especiales y restricciones.

Para poder usarla es necesario importar la librería **#include "drivers/gpio.h"**

Un ejemplo de salida podría ser:

```
=====IO DUMP Start=====
IO[4] -
  Pullup: 1, Pulldown: 0, DriveCap: 2
  InputEn: 1, OutputEn: 0, OpenDrain: 0
  FuncSel: 1 (GPIO)
  GPIO Matrix SigIn ID: (simple GPIO input)
  SleepSelEn: 1

IO[18] -
  Pullup: 0, Pulldown: 0, DriveCap: 2
  InputEn: 0, OutputEn: 1, OpenDrain: 0
  FuncSel: 1 (GPIO)
  GPIO Matrix SigOut ID: 256 (simple GPIO output)
  SleepSelEn: 1

IO[26] **RESERVED** -
  Pullup: 1, Pulldown: 0, DriveCap: 2
  InputEn: 1, OutputEn: 0, OpenDrain: 0
  FuncSel: 0 (IOMUX)
  SleepSelEn: 1

=====IO DUMP End=====
```

Esta es una salida real la función *gpio_dump_io_configuration()* mateado sobre los GPIO's 4, 18 y 26.

Lo que nos está mostrando es la configuración de cada GPIO, tenemos información como

-pullup activado (pullup – 1 : pulldown: 0)

-drivecap (drive capacity nivel 2 - ~20mA): es un parámetro que controla cuánta corriente puede soportar o drenar un pin GPIO, siendo 2 cercano a los 20mA la configuración estándar.

-inputEn configurado como entrada (inputEn : 1, output:0) nos señala que el GPIO es de entrada

-funcSel: 1 (indica que función desempeña en este caso como GPIO, pero tenemos como el caso del PIN 26 de uso directo del multiplexo I/O que es 0 y de 2-5 funciones alternativas.)

-sleepSelEn: 1 (este parámetro indica si la configuración se mantiene durante el Deep sleep, cuando esta en 1, tanto pull-ups/pull-down/ configuración de los GPIO's se mantiene pero en el caso que sea **0** estos se recetean)

-desarrollo de la practica

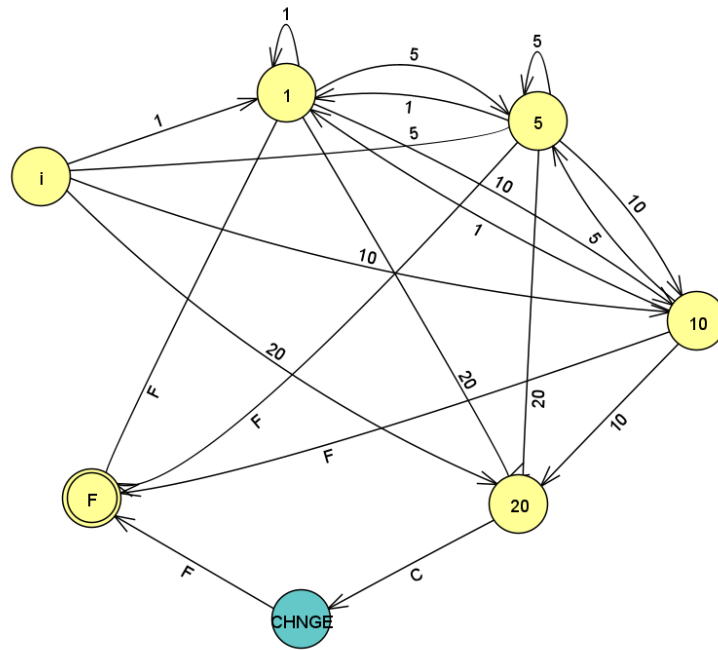
-parte 1:

Una plaza comercial cobrará 15 pesos por el uso de su estacionamiento. Cuando un conductor quiere retirar su vehículo, págase ese monto por medio de una máquina expendedora, esta le provee el recibo de pago que el conductor después ingresará en la barrera de control vehicular para poder salir de la plaza. La máquina expendedora solo permite el pago con monedas de 1, 5, 10 y 20 pesos.

Diseñe una máquina de estado que implemente el comportamiento de la máquina expendedora de recibos.

Diferentes estados en los que puede estar:

- Estado inicial
- Estado donde se insertan monedas de 1 peso
- Estado donde se insertan monedas de 5 pesos
- Estado donde se inserta monedas de 10 pesos
- Monedas donde se inserta monedas de 20 pesos
- Estado donde se pago exacto y da recibo
- Estado donde se pago de mas y requiere cambio (donde es mas de 15 pesos)



Simulación en wokwi:

estado inicial – aún no se registra nada:

main.c diagram.json parking.h Library Manager

```

720 void lower_needle_task(void *params){
725     if(current_barrier_state == BARRIER_LOWERING && current_state_needle== STATE_U
726         current_state_needle= STATE_MIDDLE;
727         update_needle_leds(STATE_MIDDLE,false); //decentso
728         printf("aguja posicon media - LOWER");
729         vTaskDelay(2000/portTICK_PERIOD_MS);
730     }
731
732     if(current_barrier_state == BARRIER_LOWERING && current_state_needle==STATE_MI
733         // MIDDLE -> DOWN
734         current_state_needle = STATE_DOWN;
735         update_needle_leds(STATE_DOWN,false);
736         printf("aguja posicon BAJO - LOWER");
737         pyment_completed=false;
738         vehicle_waiting=false;
739         current_barrier_state= BARRIER_WAITING;
740         vTaskDelay(2000/portTICK_PERIOD_MS);
741     }
742 }
743
744
745 vTaskDelete(NULL);
746 }
747
748
749
750
751
752
753

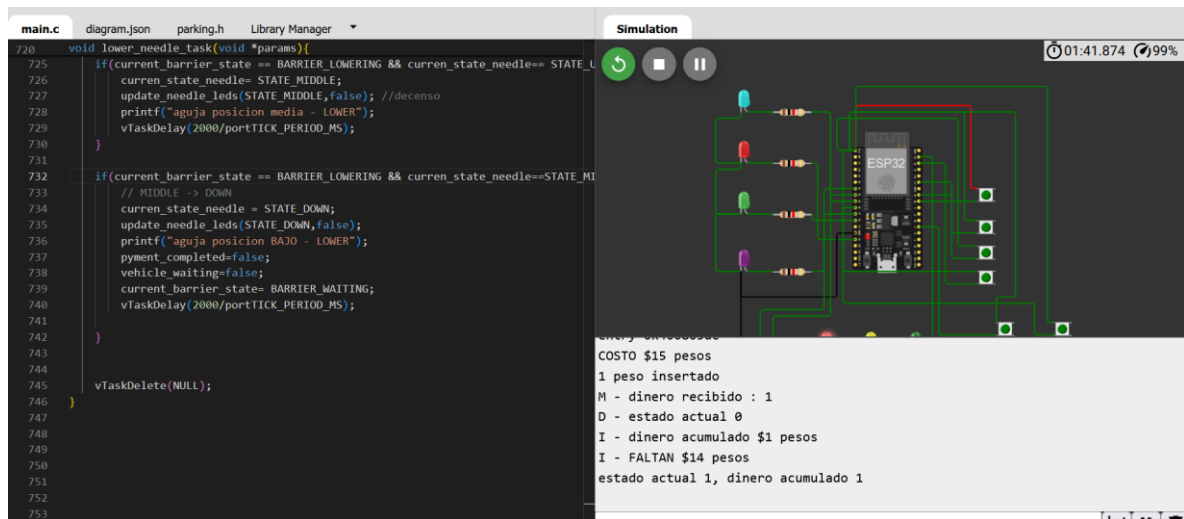
```

Simulation

00:17.836 98%

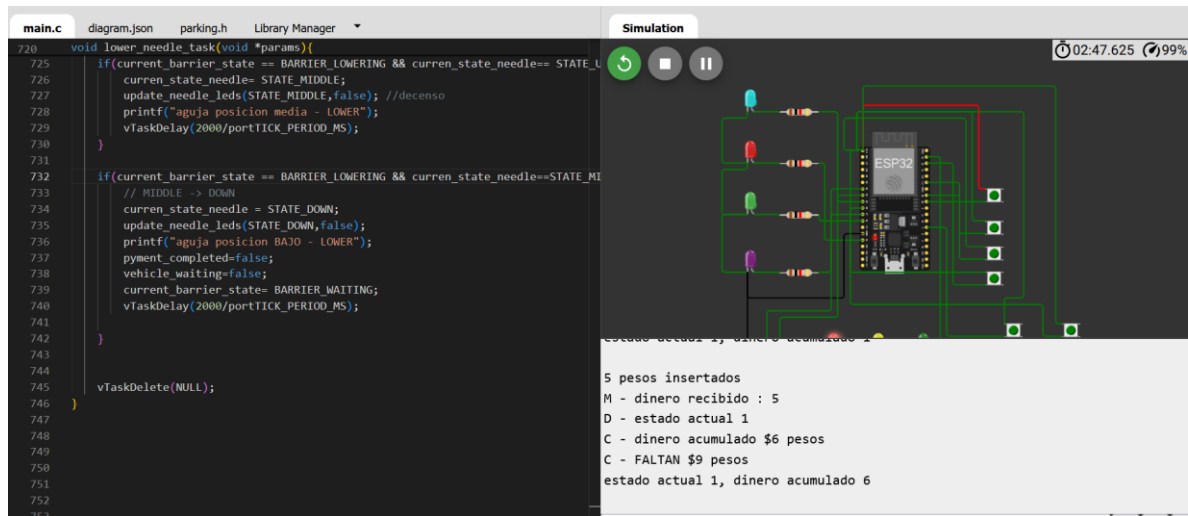
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
COSTO \$15 pesos

Insertamos moneda de 1 peso:



En la parte de abajo indico cuanto falta en caso de que no se hayan completado los 15 pesos y el dinero acumulado.

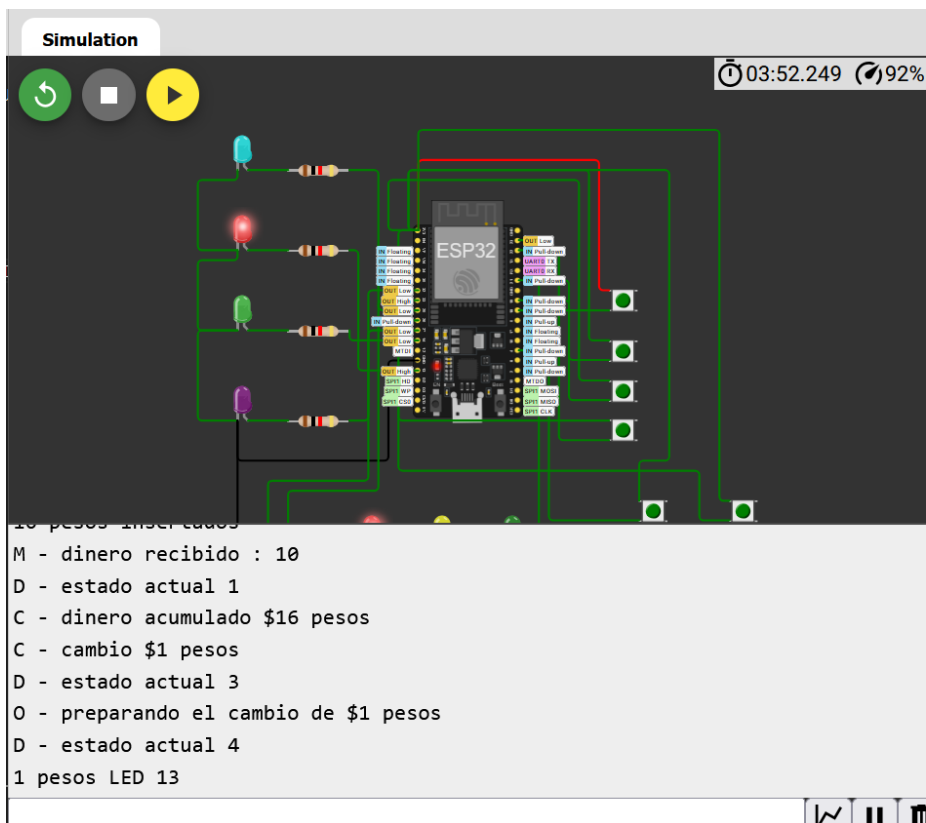
-insertar moneda de 5 pesos:



Insertar moneda de 10 pesos y recibiremos 1 de cambio

Simulation

03:52.249 92%



10 pesos insertados

```
M - dinero recibido : 10
D - estado actual 1
C - dinero acumulado $16 pesos
C - cambio $1 pesos
D - estado actual 3
O - preparando el cambio de $1 pesos
D - estado actual 4
1 pesos LED 13
```

El cambio esta representado por los leds del lado izquierdo en donde el led morado representa los de 10 pesos, el led verde de 5 y el led rojo de 1, el led azul representa el impresión del boleto, en este caso se esta dado 1 peso de cambio.

-impresión del boleto de salida:



Cada estado de led se mantiene así prendido por 2 segundos para poderlo visualizarlo correctamente.

-insertar 20 pesos

Simulation

00:03.448 97%

```
D - estado actual 0
I - dinero acumulado $20 pesos
I - cambio $5 pesos
D - estado actual 3
O - preparando el cambio de $5 pesos
D - estado actual 4
5 pesos LED 14
```

En el caso de insertar 20 pesos pasa directamente a dar el cambio y después a imprimir el boleto, en este caso el led verde representa los 5 pesos de cambio

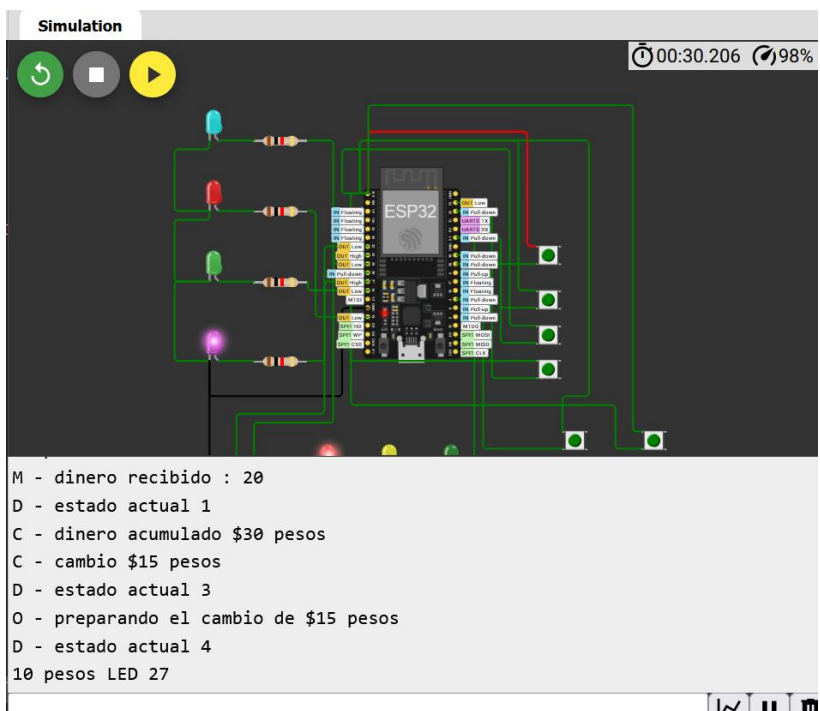
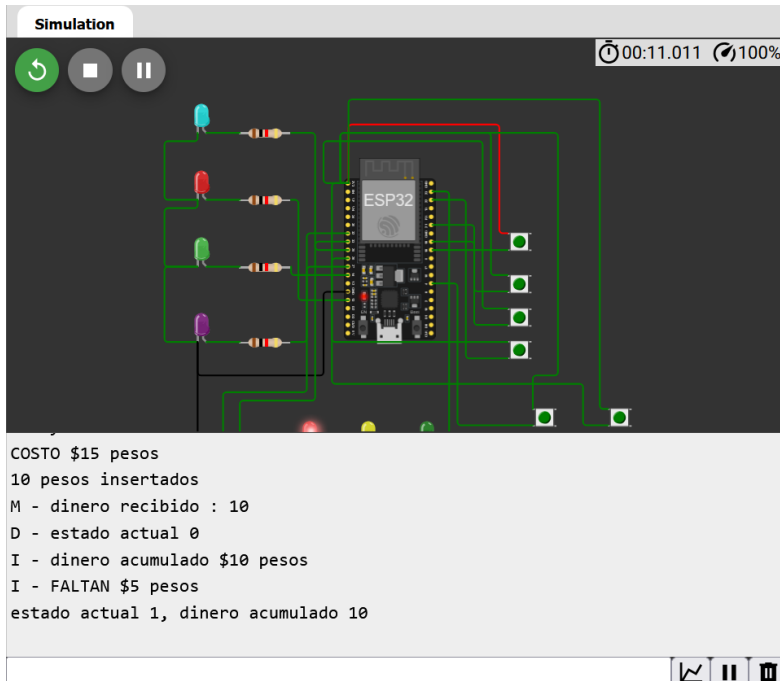
Impresión del boleto – 20 – cambio – impresión

00:06.979 100%

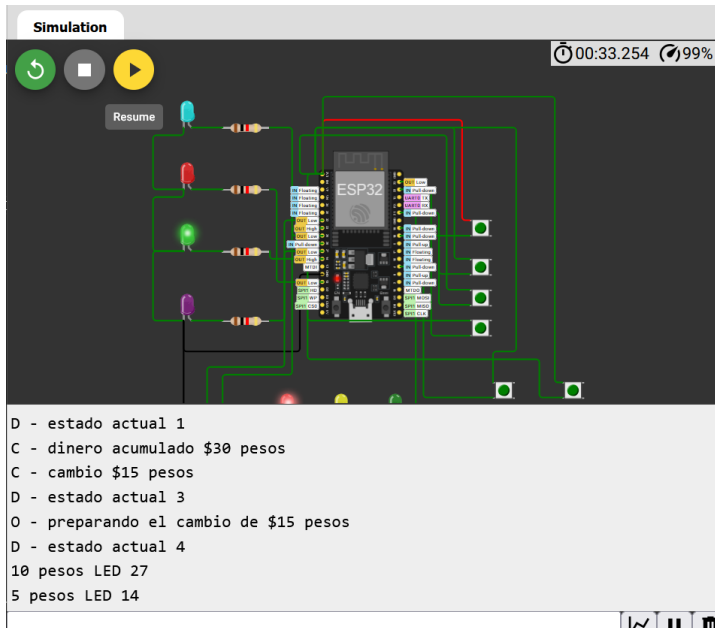
```
I - cambio $5 pesos
D - estado actual 3
O - preparando el cambio de $5 pesos
D - estado actual 4
5 pesos LED 14
D - estado actual 2
P - imprimiendo boleto!
```

-cambio de mas:

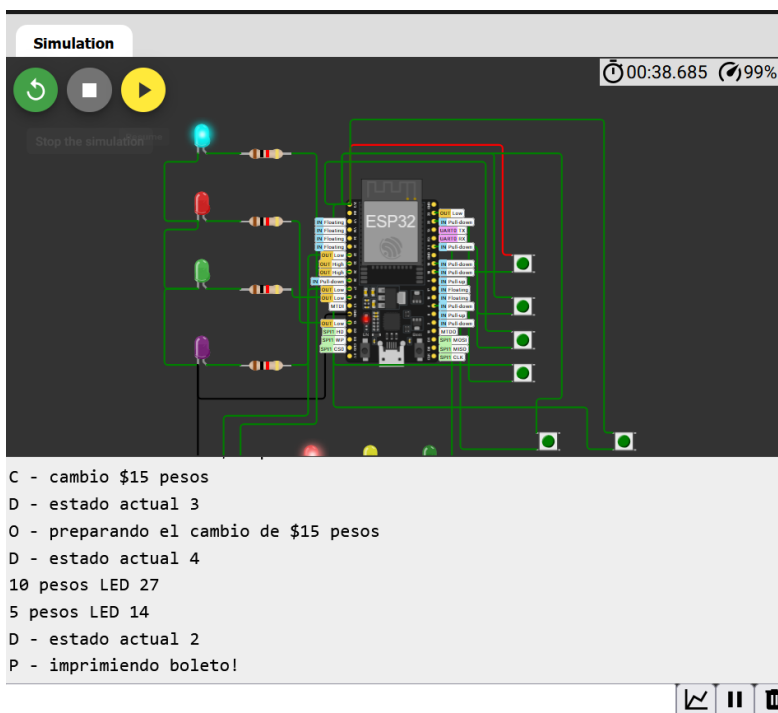
Puede dar cambio diferente a 1 peso o 5, por ejemplo si se inserta 10, requiere 5 mas pero si inserto 20 deberian de darme 15 pesos de cambio lo cual podemos ver en la siguiente representación.



Hay 30 pesos acumulados por lo que el cambio es 15, primero imprime los de mayor denominación en este caso 10 pesos representado por el led morado



Después los de 5 pesos para después pasar a dar el boleto

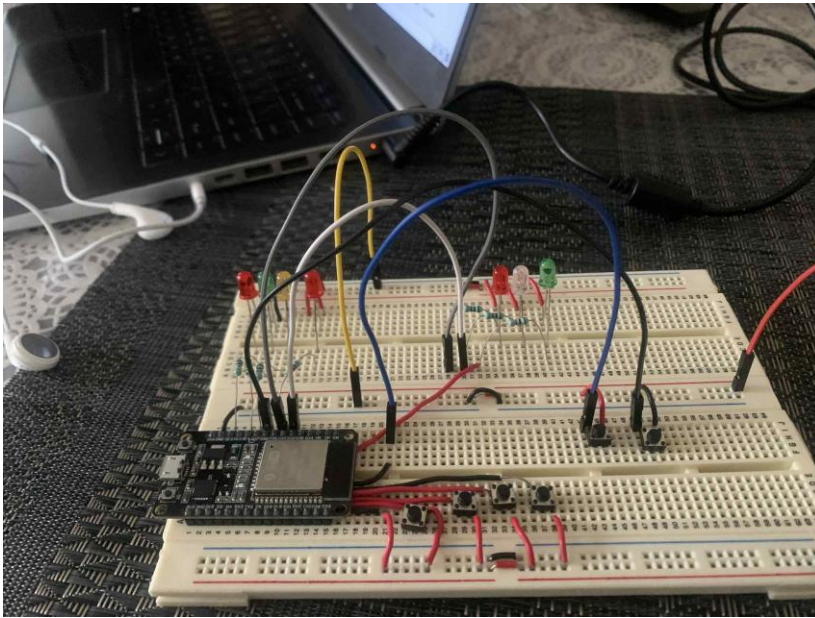


-implementación física:

Materiales:

- 1- Esp32
- 2- 6 push bottom
- 3- 7 resistencias de 1Kohms
- 4- 7 leds
- 5- protoboard

Ensamblado en el protoboard

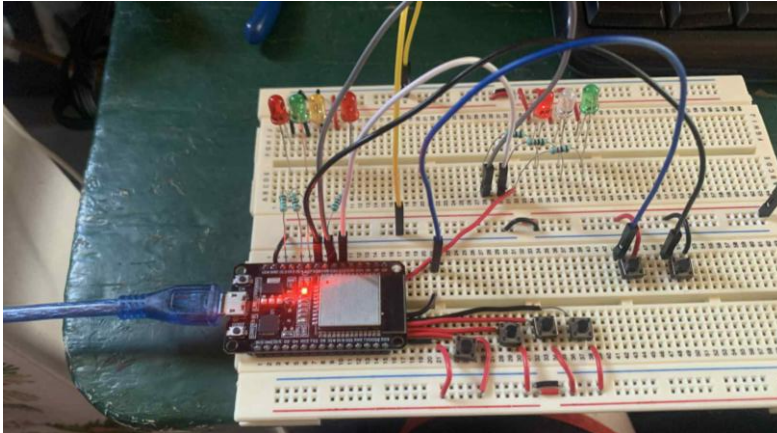


Cargar el programa al ESP32:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ESP-IDF

I (193) cpu_start: cpu freq: 160000000 Hz
I (193) app_init: Application information:
I (193) app_init: Project name:      parte_1_parking
I (198) app_init: App version:      824d5fb-dirty
I (202) app_init: Compile time:     Sep  5 2025 11:00:59
I (207) app_init: ELF file SHA256:  fa0f1e470...
I (211) app_init: ESP-IDF:         v5.5
I (215) efuse_init: Min chip rev:   v0.0
I (219) efuse_init: Max chip rev:   v3.99
I (223) efuse_init: Chip rev:      v3.1
I (227) heap_init: Initializing. RAM available for dynamic allocation:
I (233) heap_init: At 3FFAE6E0 len 00001920 (6 KIB): DRAM
I (238) heap_init: At 3FFB2D18 len 0002D2F0 (190 KIB): DRAM
I (243) heap_init: At 3FFE0440 len 000324C0 (14 KIB): D/IRAM
I (249) heap_init: At 3FFE4350 len 0001DC00 (111 KIB): D/IRAM
I (254) heap_init: At 40080614 len 000129EC (74 KIB): IRAM
I (261) spi_flash: detected chip: generic
I (263) spi_flash: flash io: dio
W (266) spi_flash: Detected size(4096k) larger than the size in the binary image header(2048k). Using the size in the binary image header.
I (279) main_task: Started on CPU0
I (280) main_task: Calling app_main()
COSTO $15 pesos
I (289) main_task: Returned from app_main()
I
```

Estado de inicio – un no se inserta nada:



- **Insertamos 1 peso:**

por medio de la consola vemos que se ha insertado el peso correctamente

```
I (249) heap_init: At 3FFE4350 len 0001BCB0 (111 KiB): D/IRAM  
COSTO $15 pesos  
I (289) main_task: Returned from app_main()  
1 peso insertado  
M - dinero recibido : 1  
D - estado actual 0  
I - dinero acumulado $1 pesos  
I - FALTAN $14 pesos  
estado actual 1, dinero acumulado 1
```

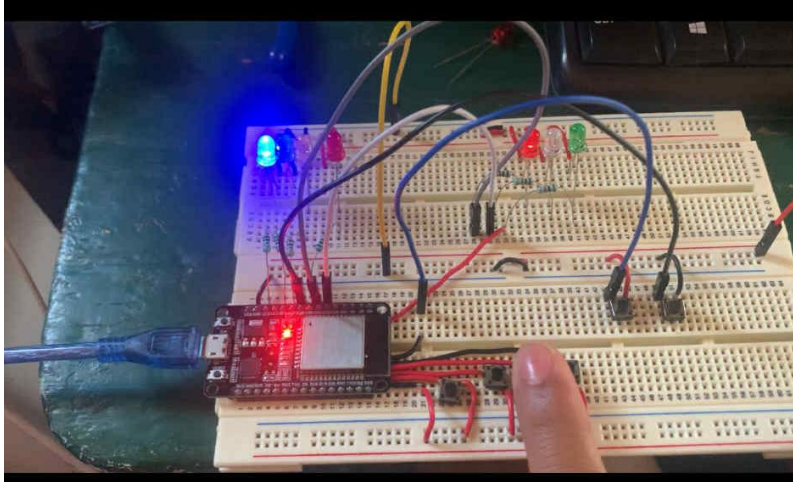
- **Insertamos 5 pesos**

```
I (249) heap_init: At 3FFE4350 len 0001BCB0 (111 KiB): D/IRAM  
I (254) heap_init: At 4008D614 len 000129EC (74 KiB): IRAM  
I (261) spi_flash: detected chip: generic  
I (263) spi_flash: flash io: dio  
W (266) spi_flash: Detected size(4096k) larger than the size in the  
I (279) main_task: Started on CPU0  
I (289) main_task: Calling app_main()  
COSTO $15 pesos  
I (289) main_task: Returned from app_main()  
1 peso insertado  
M - dinero recibido : 1  
D - estado actual 0  
I - dinero acumulado $1 pesos  
I - FALTAN $14 pesos  
estado actual 1, dinero acumulado 1  
  
5 pesos insertados  
M - dinero recibido : 5  
D - estado actual 1  
C - dinero acumulado $6 pesos  
C - FALTAN $9 pesos  
estado actual 1, dinero acumulado 6
```

Al igual que en la simulación me indica el dinero que inserte y lo que me falta,

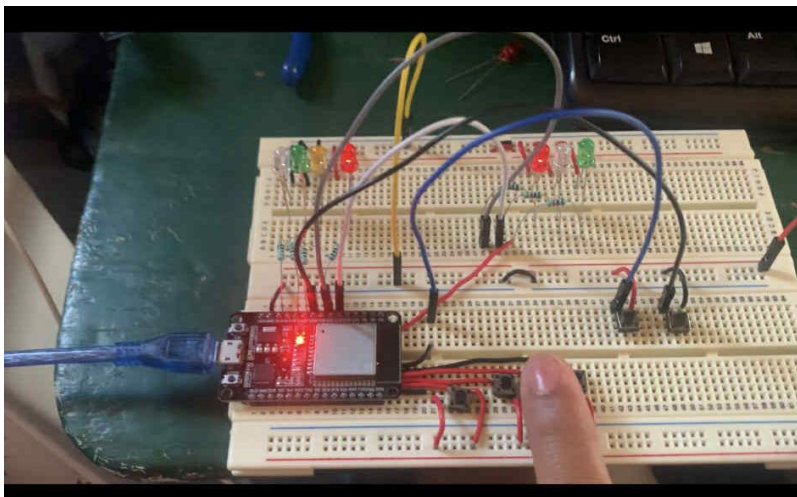
- **Insertar moneda de 10:**

al insertar la moneda de 10 me da el cambio que es de 1 peso



El led azul indica las monedas de 1 peso, el led verde las de 5, el amarillo de 10 y el led rojo indica que se imprimio el boleto

- **Imprime el boleto**



Salida por monitor:

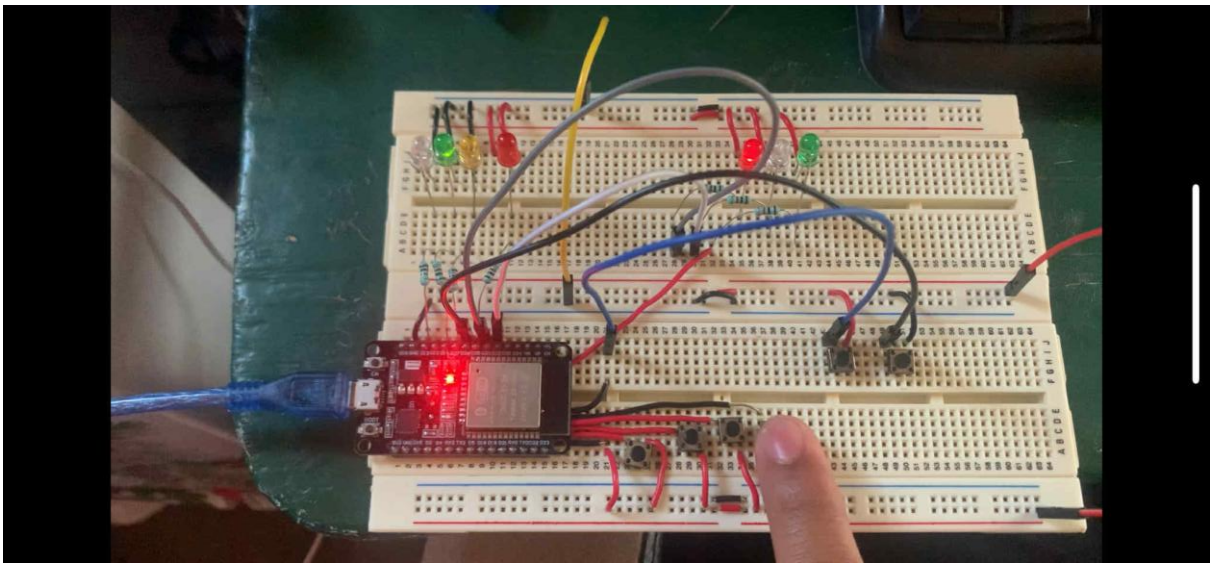
```
10 pesos insertados
M - dinero recibido : 10
D - estado actual 1
C - dinero acumulado $16 pesos
C - cambio $1 pesos
D - estado actual 3
O - preparando el cambio de $1 pesos
D - estado actual 4
1 pesos LED 13
D - estado actual 2
P - imprimiendo boleto!
P - dinero acumulado $16 pesos
D - estado actual 5
R - TRANSACCION COMPLETADO
Buen viaje
listo para procesar al vehiculo
estado actual 0, dinero acumulado 0
```

- Insertar moneda de 20 pesos

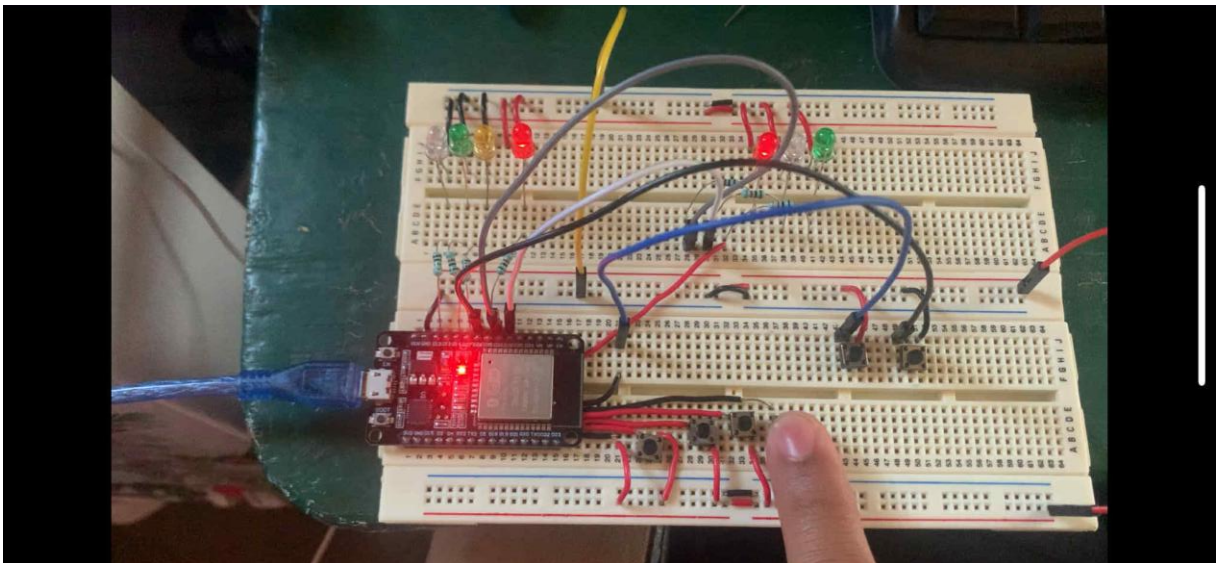
Recibe 20 pesos:

```
M - dinero recibido : 20
D - estado actual 0
I - dinero acumulado $20 pesos
I - cambio $5 pesos
D - estado actual 3
O - preparando el cambio de $5 pesos
D - estado actual 4
5 pesos LED 14
D - estado actual 2
P - imprimiendo boleto!
P - dinero acumulado $20 pesos
D - estado actual 5
R - TRANSACCION COMPLETADO
Buen viaje
listo para procesar al vehiculo
estado actual 0, dinero acumulado 0
```

Salida por el led verde indicando cambio de 5 pesos



Imprime el boleto:



- **Cambio de mas:**

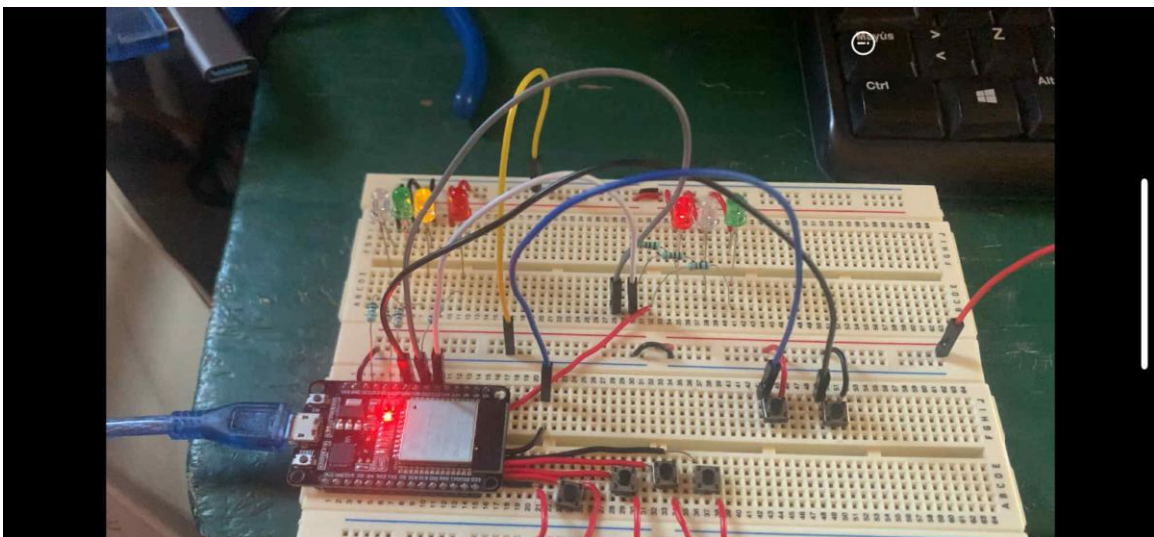
Ahora insertaremos 30 pesos, primero 10 y después 20 para que indique mas estados en el cambio

Salida en monitor:

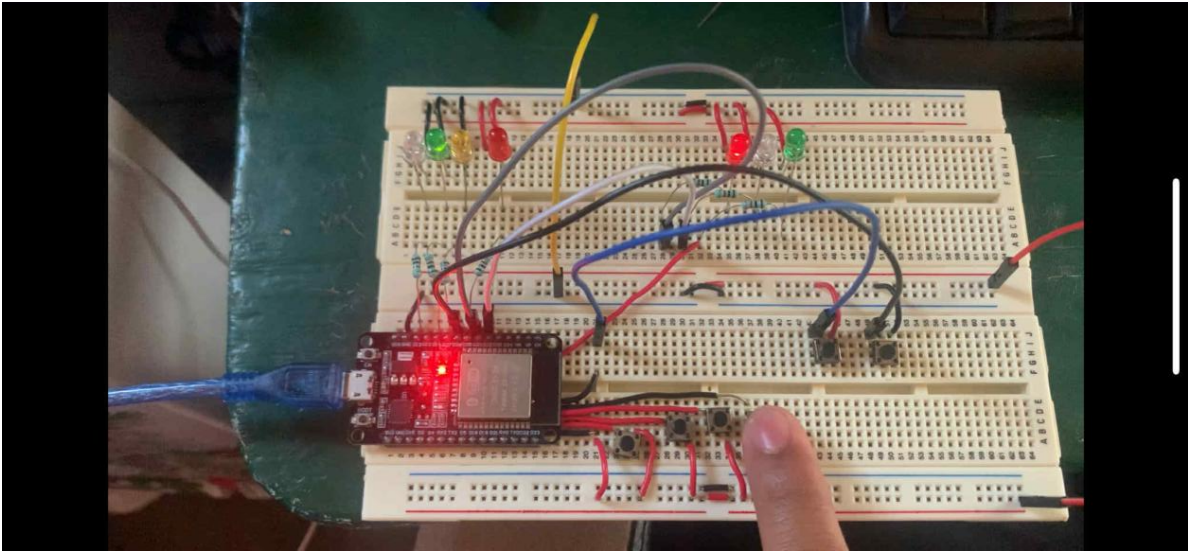
```
10 pesos insertados
M - dinero recibido : 10
D - estado actual 0
I - dinero acumulado $10 pesos
I - FALTAN $5 pesos
estado actual 1, dinero acumulado 10

20 pesos insertados
M - dinero recibido : 20
D - estado actual 1
C - dinero acumulado $30 pesos
C - cambio $15 pesos
D - estado actual 3
O - preparando el cambio de $15 pesos
D - estado actual 4
10 pesos LED 27
5 pesos LED 14
D - estado actual 2
P - imprimiendo boleto!
P - dinero acumulado $30 pesos
D - estado actual 5
R - TRANSACCION COMPLETADO
Buen viaje
listo para procesar al vehiculo
estado actual 0, dinero acumulado 0
```

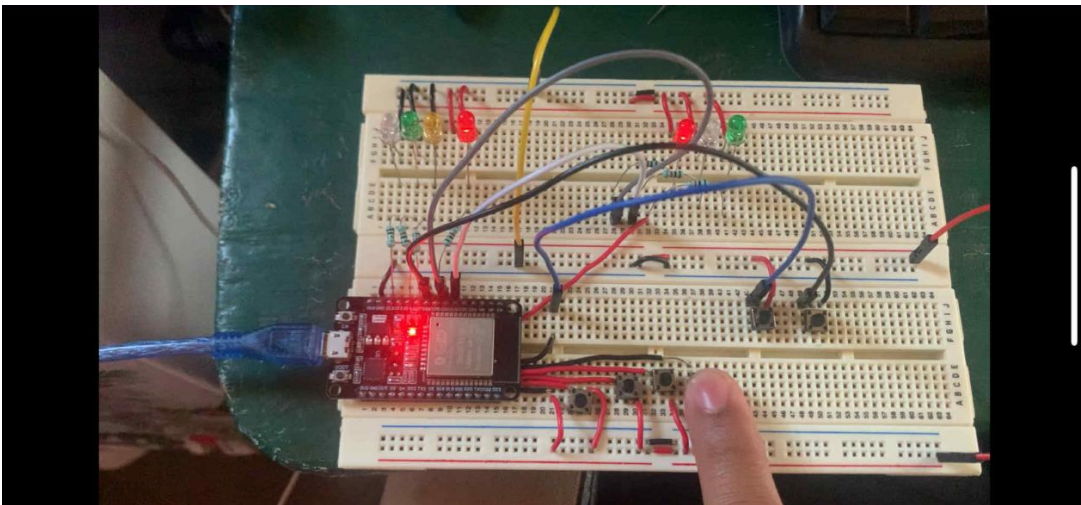
-cambio de 10 pesos – led amarillo



-cambio de 5 pesos:



-impresión de boleto:



Parte 2:

Disene e implemente el comportamiento de lá bárrerá de control vehiculár por medio de una máquina de estados. Lás ácciones que debe reálizár son:

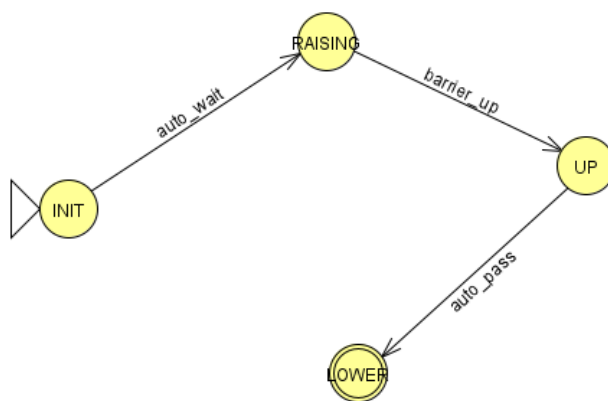
- Si háy un cárro en esperá de cruzár lá bárrerá de control vehiculár, está tiene que elevár la águjá hástá que este en lá posicion de árribá.
- Uná vez árribá, debe permánecer así hástá que el cárro haya cruzado lá bárrerá.
- Después de que el carro ha cruzado, la barrerá de control vehicular tiene que bajar la aguja hasta la posición de abajo.

la aguja se desplaza en 3 posiciones **abajo, en medio y arriba**, la aguja debe de pasar por esos 3 estados cunado se eleva y de manera inversa cunado esta arriba y requiere descender.

Estados donde podría estar la barrea de control:

- Estado inicial donde está en espera de un vehículo
- Estado donde se está elevando la aguja
- Estado donde la guja ya está en la posición de arriba y la barrea de control vehiculas esta en espera de que el carro pase
- Estado donde está bajando la guja

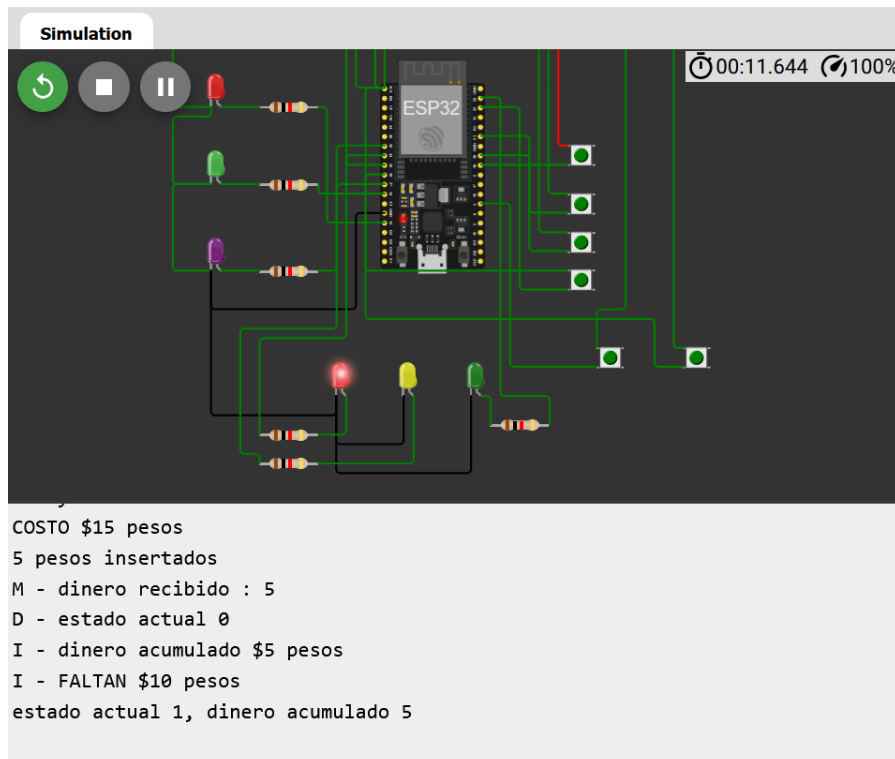
diagrama de estados para la barrea:



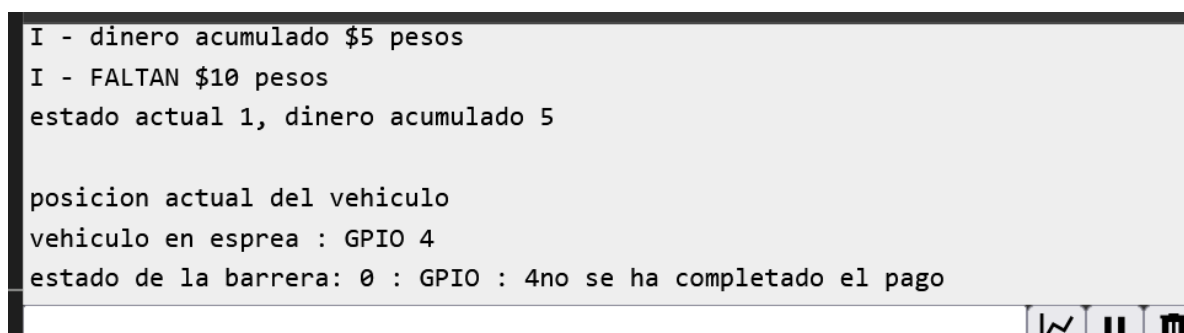
Simulación en wokwi:

Para que el proceso de la animación de elevación de la aguja primero debe de dar la señal que se completo el pago en la caseta de otro modo no se deja

Primero inserto 5 pesos:



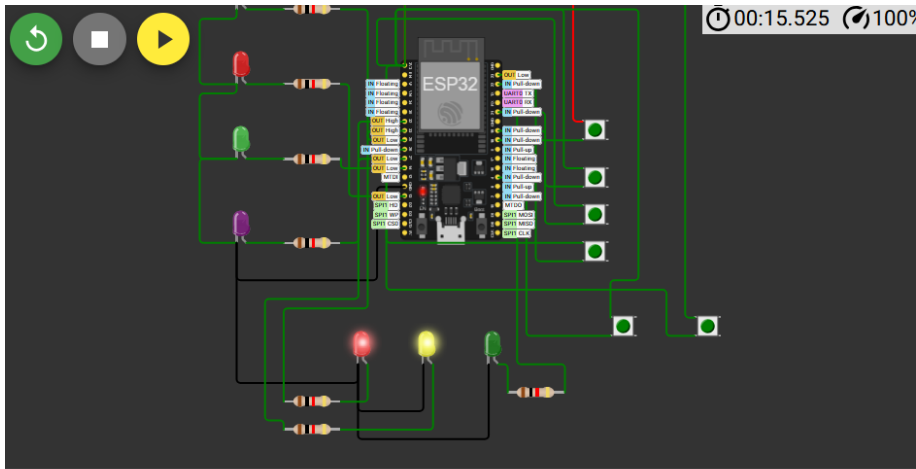
Después quiero indicar que estoy esperando lo cual me lanzara que un no se ha completado el pago



Realizo el pago completo y presiono el botón izquierdo que indica que el **vehículo esta en espera** en espera que se eleve la aguja.

En un inicio esta establecido un led rojo que indica que la barrera esta abajo.

Ahora está en posición de **en medio**



listo para procesar al vehiculo
estado actual 0, dinero acumulado 0

posicion actual del vehiculo

vehiculo en esprea : GPIO 4

estado de la barrera: 0 : GPIO : 4vehiculo detectado - inicacion elevacion de
barrea

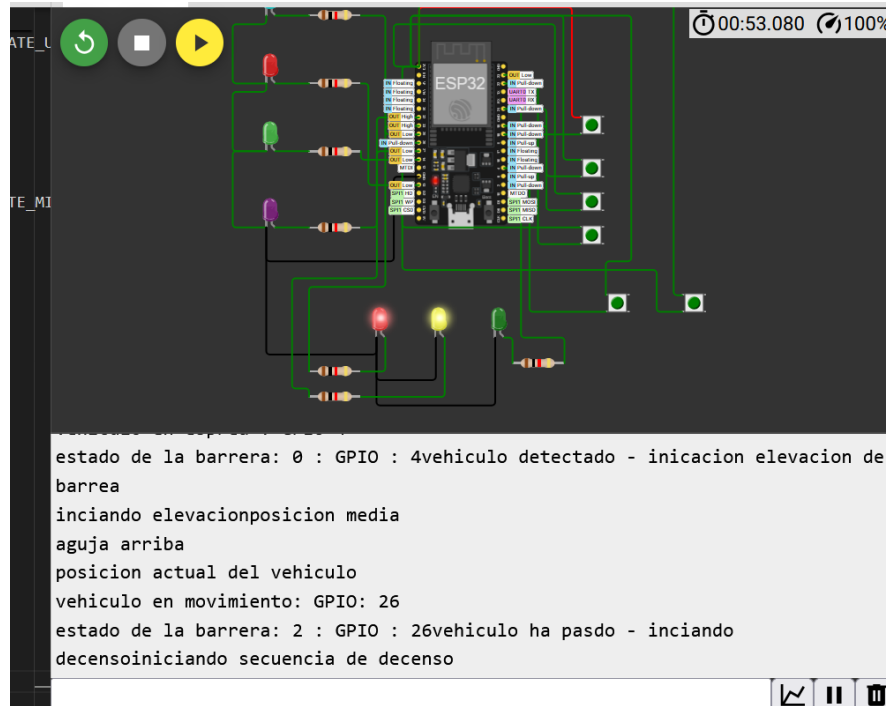
inciando elevacionposicion media

Después la aguja está completamente arriba:

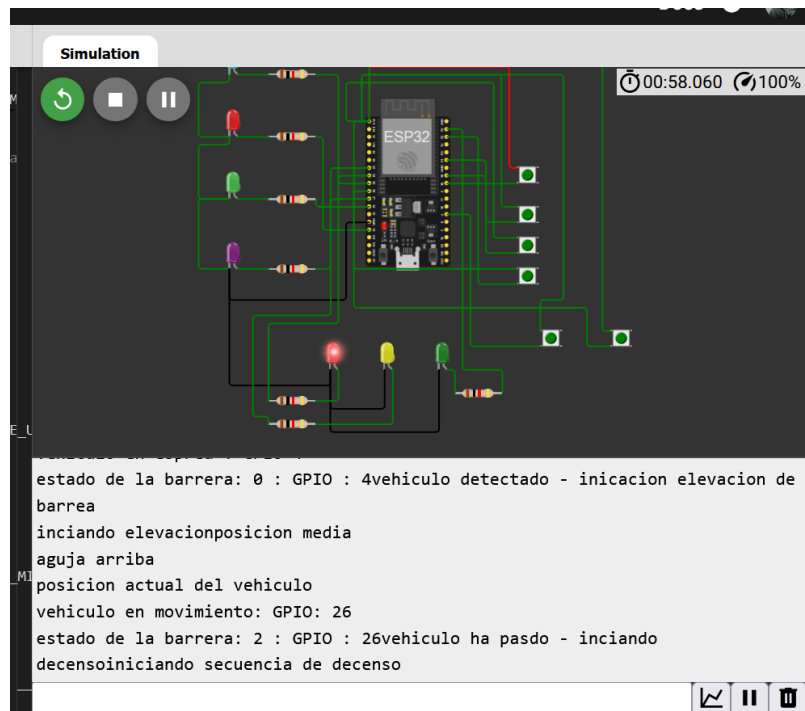


Se quedará así hasta que presiono el botón de la derecha indicando que **el vehículo paso** lo que bajara la aguja

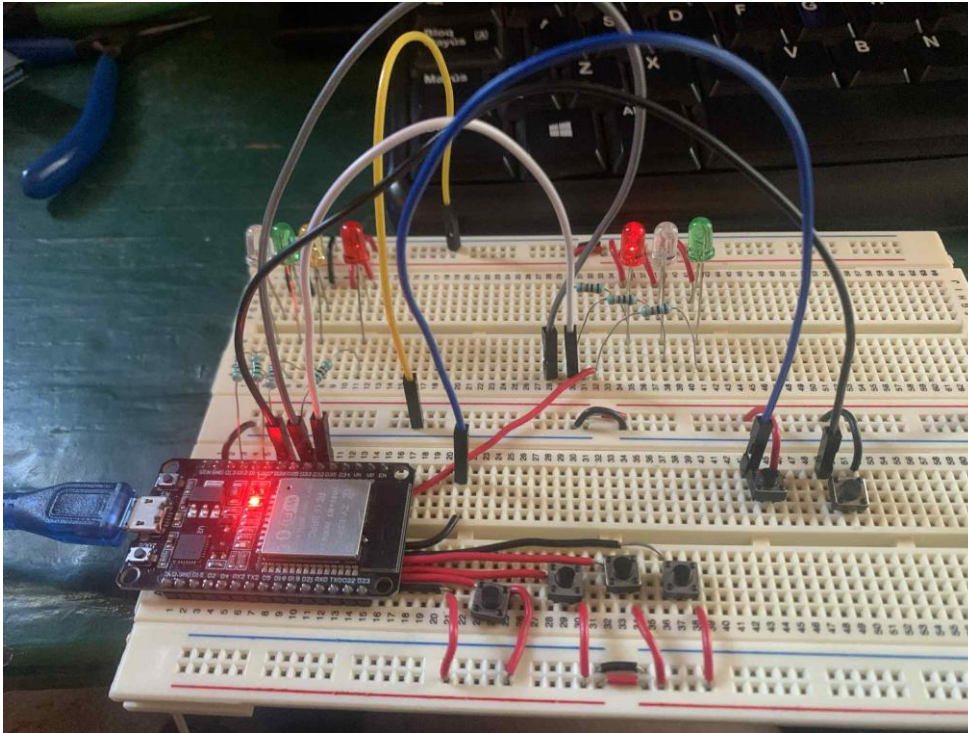
Ahora está en posición de **en medio**:



Ahora ya está completamente **abajo** por lo que esta listo para recibir otro vehículo



Implementación en físico:



el led rojo indica que esta abajo la aguja inicia encendido, el botón de la izquierda indica que es carro está en espera.

Si quiero indicar que estoy en espera para que me de salida me dirá que aun no se ha completado el pago.

```
I (254) heap_init: At 4008D614 len 000129EC (74 KiB): IRAM
I (261) spi_flash: detected chip: generic
I (263) spi_flash: flash io: dio
W (266) spi_flash: Detected size(4096k) larger than the size in the binary image header.
I (279) main_task: Started on CPU0
I (289) main_task: Calling app_main()
COSTO $15 pesos
I (289) main_task: Returned from app_main()
posicion actual del vehiculo
vehiculo en espera : GPIO 4
estado de la barrera: 0 : GPIO : 4
no se ha completado el pago
barrera cerrada
```

Y no habrá cambio

- Insertamos dinero y pedimos salida

Para que me de la salida, insertamos el dinero suficiente entonces el sistema estará listo para recibir la señal

```
I - FALTAN $5 pesos
estado actual 1, dinero acumulado 10

5 pesos insertados
M - dinero recibido : 5
D - estado actual 1
C - dinero acumulado $15 pesos
C - pago completado
D - estado actual 2
P - imprimiendo boleto!
P - dinero acumulado $15 pesos
D - estado actual 5
R - TRANSACCION COMPLETADO
Buen viaje
listo para procesar al vehiculo
estado actual 0, dinero acumulado 0
```

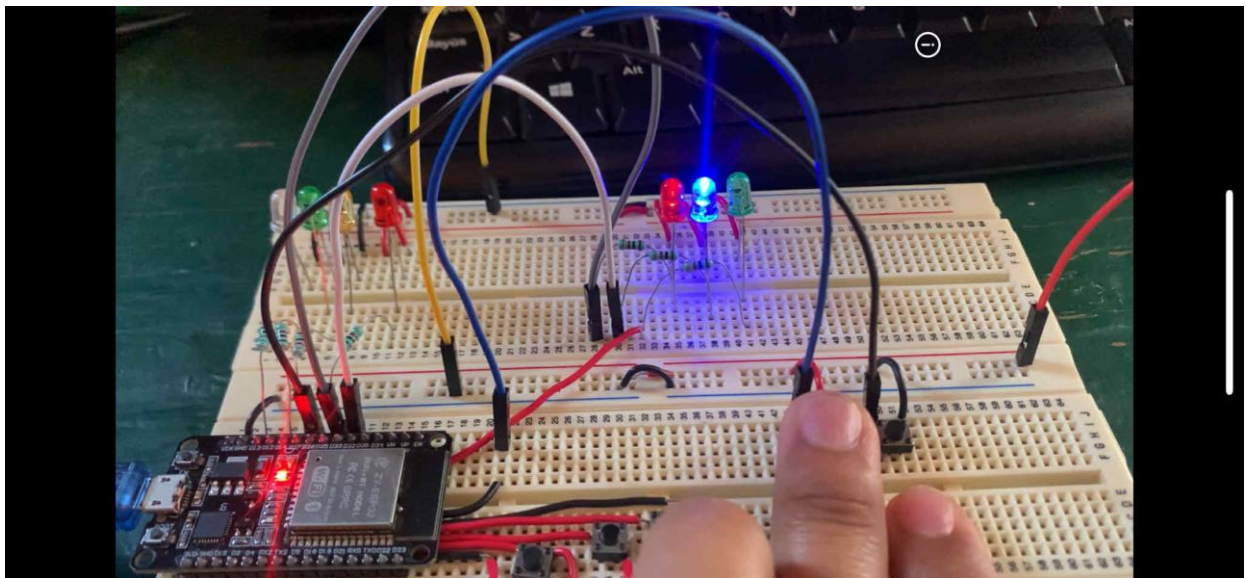
en monitor me indica que está listo para procesar al vehículo.

Indico que el vehículo está en espera y empieza a abrir la aguja:

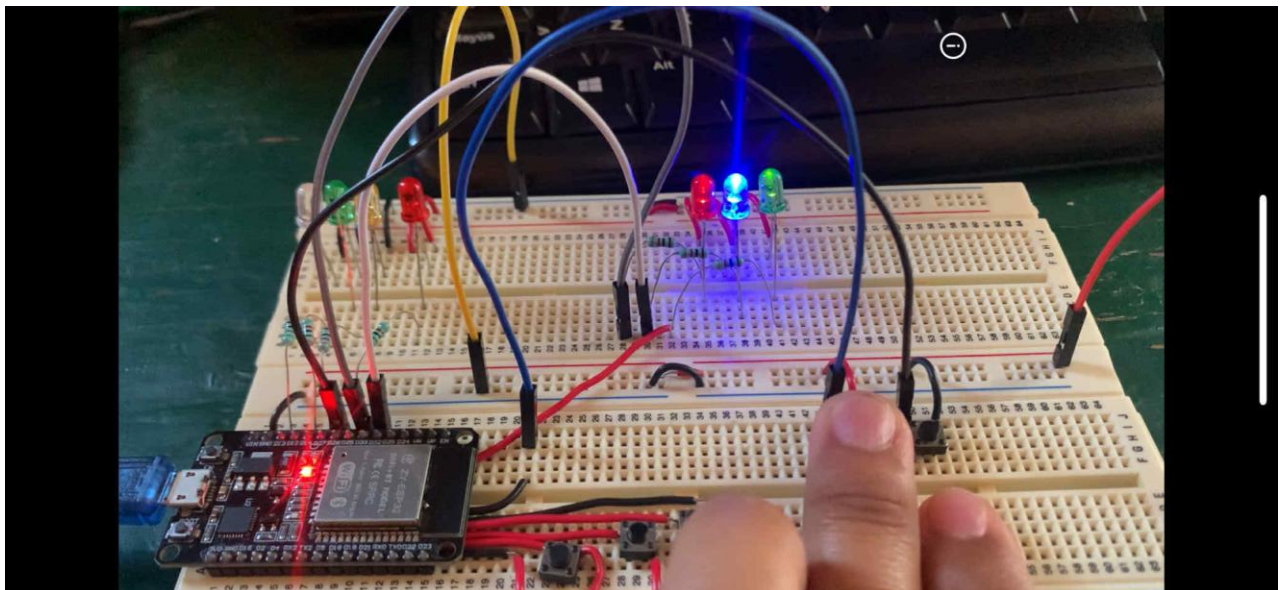
```
listo para procesar al vehiculo
estado actual 0, dinero acumulado 0

posicion actual del vehiculo
vehiculo en espera : GPIO 4
estado de la barrera: 0 : GPIO : 4
vehiculo detectado - iniciacion elevacion de barrea
iniciando elevacion
posicion media
aguja arriba
```

Posición en los leds, **en medio**



Posición **arriba**:

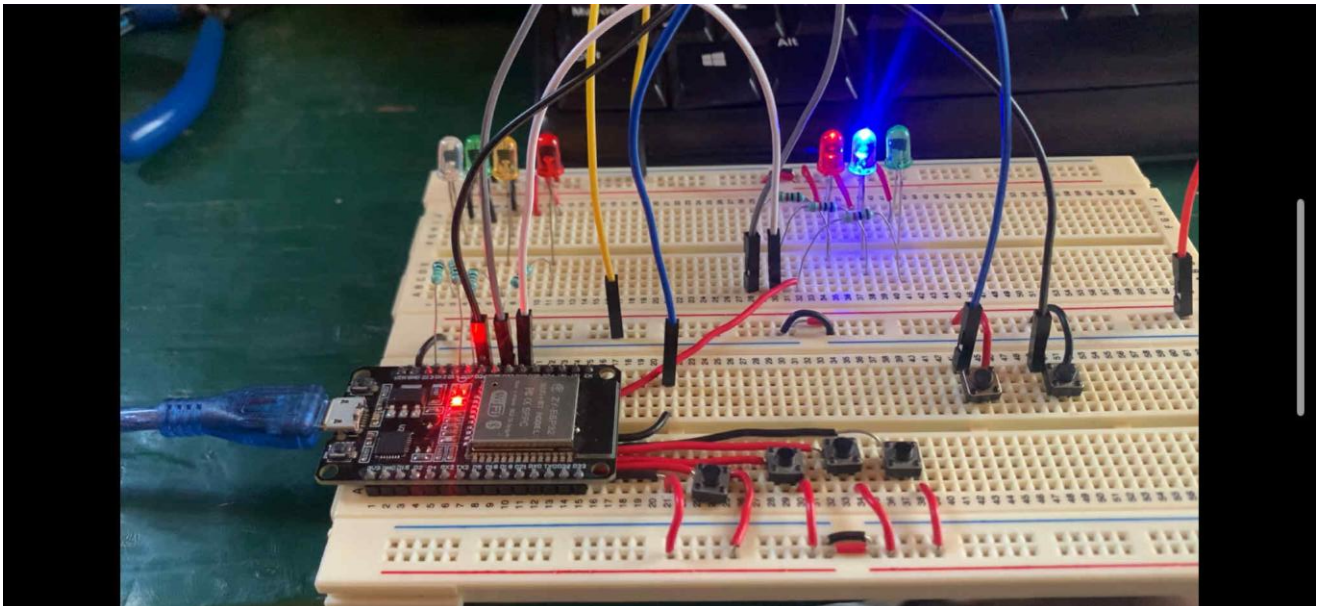


El vehículo ya paso por lo que el botón derecho indica que el vehículo paso y ahora regresa

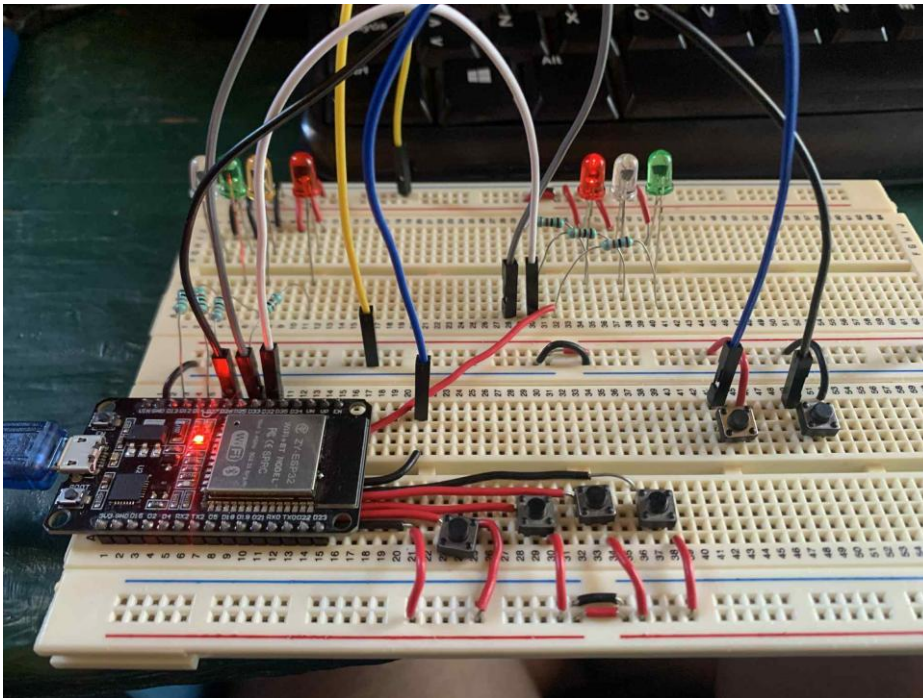
En monitor indica el descenso:

```
posicion actual del vehiculo
vehiculo en esprea : GPIO 4
estado de la barrera: 0 : GPIO : 4
vehiculo detectado - inicacion elevacion de barrea
inciando elevacion
posicion media
aguja arriba
posicion actual del vehiculo
vehiculo en movimiento: GPIO: 26
estado de la barrera: 2 : GPIO : 26
vehiculo ha pasdo - inciando decenso
inciando secuencia de decenso
aguja posicion media - LOWER
aguja posicion BAJO - LOWER
```

Apagando la posición **arriba**:



Apagando la posición de **en medio**:



Conclusiones y comentarios:

Como conclusión considero que la practica es muy buena como introducción porque es un cambio de como se programa en ESP32 al ATmega2560, ya que estoy acostumbrado en configurar varias cosas para que un GPIO sea de entrada o de salida y con el SDK del IDF solo se ocupa una función y hace todo el trabajo, con esta practica se pudo trabajar de manera intensa con los componentes básicos que son los GPIOs y conocer cuales pines están disponibles y cuales no.

Dificultades en el desarrollo:

Tuve varios problemas en el desarrollo más que nada en conceptos nuevos como las colas y las tareas, fue lo más se me dificultó en tratar de entender no se si debería de haberlas implementarlas pero el cómo declararlas, sus parámetros no me quedaban del todo claro, tuve varios problemas sobre como establecer las características que necesitaba para los GPIO's de entrada, pero de ahí en fue no tuve problemas más que de lógica o bugs de estados que se pasaban o no contaba, pero creo que mis dificultades pueden resolverse estudiando más.

Referencias:

<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/gpio.html>

https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos_idf.html

<https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/02-Queues-mutexes-and-semaphores/01-Queues>

<https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/01-Tasks-and-co-routines/02-Task-states>